Saga

aga · Order: reserve→charge→ship with reverse compensations · Build with Waqa

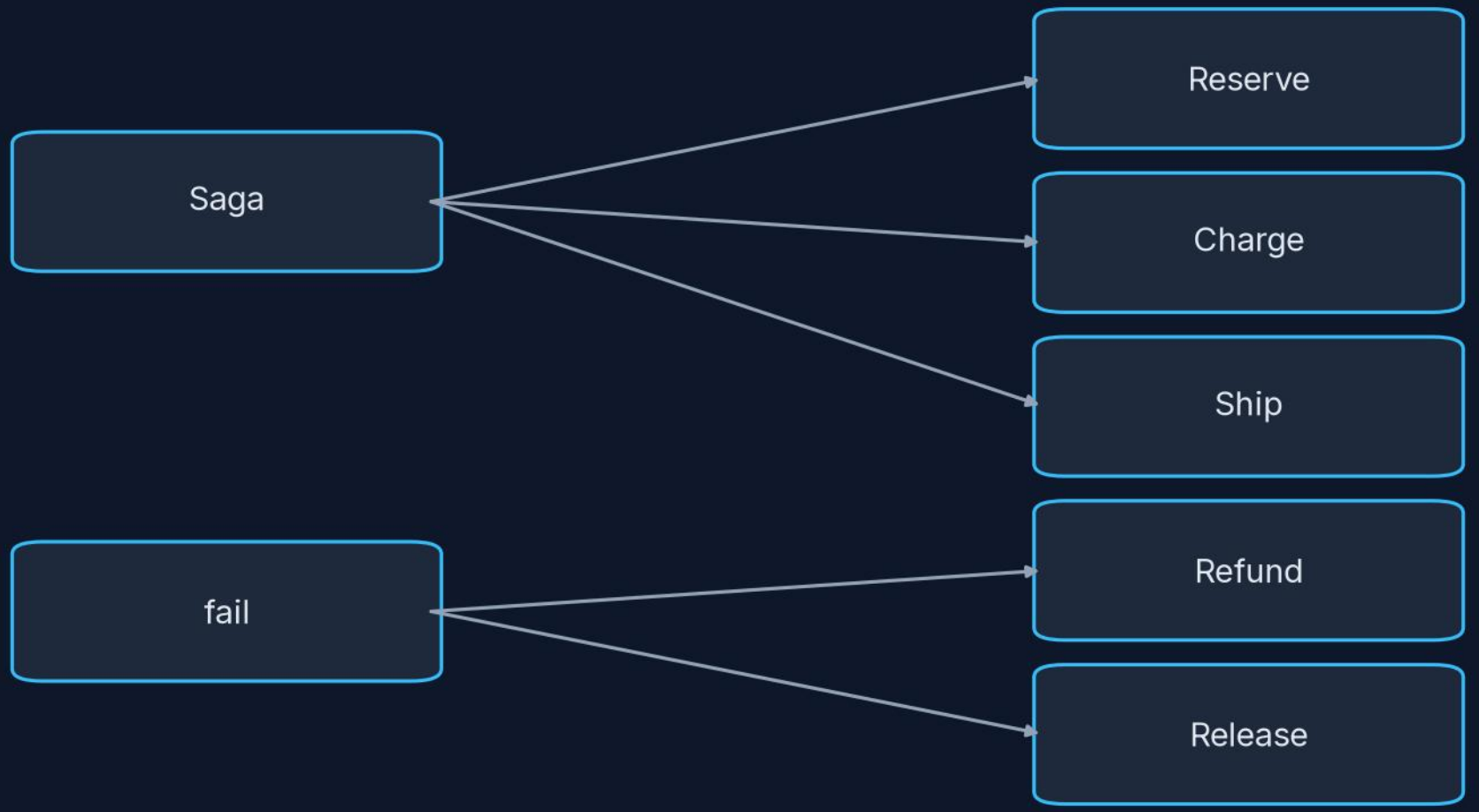
Today's Hook

- Problem: Order: reserve→charge→ship with reverse compensations
- Pattern: Saga — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Order: reserve→charge→ship with reverse compensations
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Saga: Order: reserve→charge→ship with reverse compensati

Real-World Scenario

- Running example: Order: reserve→charge→ship with reverse compensations
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

/**
 * Naive: sequential calls with no compensation registry.
 * If shipping fails after payment, money is taken and inventory stays reserved.
 */
public final class NaiveOrderFlow {
    private final OrderBookingSaga.InMemoryPartners partners;

    public NaiveOrderFlow(OrderBookingSaga.InMemoryPartners partners) {
        this.partners = partners;
    }

    public boolean book(String orderId, String sku, int qty, int cents, String address) {
        var r1 = partners.reserve(orderId, sku, qty);
        if (!r1.ok()) return false;
        var r2 = partners.charge(orderId, cents);
        if (!r2.ok()) return false; // inventory still reserved — leak
        var r3 = partners.ship(orderId, address);
        return r3.ok(); // payment charged even if ship fails — leak
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.function.Supplier;

/**
 * Orchestration saga for: reserve inventory → charge payment → create shipment.
 * Each step registers a compensation; on failure compensations run in reverse order.
 * Steps are idempotent via a simple seen-set keyed by orderId+step.
 */
public final class OrderBookingSaga {
    public record StepResult(boolean ok, String detail) {}

    public interface Inventory { StepResult reserve(String orderId, String sku, int qty); void release(String orderId); }
    public interface Payments { StepResult charge(String orderId, int cents); void refund(String orderId); }
    public interface Shipping { StepResult ship(String orderId, String address); void recall(String orderId); }

    private final Inventory inventory;
    private final Payments payments;
    private final Shipping shipping;
    private final List<String> log = new ArrayList<>();
    private final List<String> seen = new ArrayList<>();
```

Solution (2)

```
public OrderBookingSaga(Inventory inventory, Payments payments, Shipping shipping) {
    this.inventory = inventory;
    this.payments = payments;
    this.shipping = shipping;
}

public List<String> log() { return List.copyOf(log); }

public boolean book(String orderId, String sku, int qty, int cents, String address) {
    List<Runnable> compensations = new ArrayList<>();
    try {
        runOnce(orderId + ":reserve", () -> {
            var r = inventory.reserve(orderId, sku, qty);
            if (!r.ok()) throw new IllegalStateException("inventory:" + r.detail());
            log.add("ok:reserve");
            compensations.addFirst(() -> { inventory.release(orderId); log.add("compensate:release"); });
        });
        runOnce(orderId + ":charge", () -> {
            var r = payments.charge(orderId, cents);
            if (!r.ok()) throw new IllegalStateException("payment:" + r.detail());
            log.add("ok:charge");
            compensations.addFirst(() -> { payments.refund(orderId); log.add("compensate:refund"); });
        });
        runOnce(orderId + ":ship", () -> {
            var r = shipping.ship(orderId, address);
            if (!r.ok()) throw new IllegalStateException("shipping:" + r.detail());
            log.add("ok:ship");
            compensations.addFirst(()
```

Runnable main

```java
package com.buildwithwaqar.patterns.enterprise.saga.problem;

/** Problem: reserve → charge → ship with reverse compensations on mid-flow failure. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Saga problem: order booking with rollback ===");
        System.out.println("--- naive (payment fails → inventory leak) ---");
        var leak = new OrderBookingSaga.InMemoryPartners(() -> true, () -> false, () -> true);
        System.out.println("naiveBooked=" + new NaiveOrderFlow(leak).book("N1", "SKU", 1, 100, "A"));
        System.out.println("naiveReserved=" + leak.reserved());

        System.out.println("--- saga happy path ---");
        var okP = OrderBookingSaga.InMemoryPartners.alwaysOk();
        var ok = new OrderBookingSaga(okP, okP, okP);
        System.out.println("booked=" + ok.book("S1", "SKU", 1, 1000, "Ship St"));
        System.out.println("log=" + ok.log());

        System.out.println("--- saga fails at ship; compensates reverse ---");
        var failP = new OrderBookingSaga.InMemoryPartners(() -> true, () -> true, () -> false);
        var fail = new OrderBookingSaga(failP, failP, failP);
        System.out.println("booked=" + fail.book("S2", "SKU", 1, 1000, "Ship St"));
        System.out.println("log=" + fail.log());
        System.out.println("reserved=" + failP.reserved() + " charged=" + failP.charged() + " shipped=" + failP.shipped());
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Saga problem: order booking with rollback ===
--- naive (payment fails → inventory leak) ---
naiveBooked=false
naiveReserved=[N1]
--- saga happy path ---
booked=true
log=[ok:reserve, ok:charge, ok:ship, committed]
--- saga fails at ship; compensates reverse ---
booked=false
log=[ok:reserve, ok:charge, abort:shipping:carrier_down, compensate:refund, compensate:release, rolled_back]
reserved=[] charged=[] shipped=[]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Saga.

Related Patterns

- Contrast Saga with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Saga → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.